

Appendix A

Proof. (Proof of Theorem 2) Line 2 scans each edge in $\mathcal{E}(\mathcal{G})$ once, therefore, the total time complexity is $\mathcal{O}(m)$. $\mathcal{G}_q$ takes $\mathcal{O}(n+m)$ space, therefore, the total space complexity is $\mathcal{O}(n+m)$.

Proof. (Proof of Theorem 3) Lines 1-2 initialize the $\mathcal{A}(u)$ and $\mathcal{D}(u)$ for each vertex $u \in \mathcal{V}(\mathcal{G})$, which takes $\mathcal{O}(n)$ time. Line 6 scans each edge in $\mathcal{E}(\mathcal{G})$ once, which takes $\mathcal{O}(m)$ time. Therefore, the total time complexity is $\mathcal{O}(n+m)$.

Throughout the algorithm, we maintain $\mathcal{A}(u)$, $\mathcal{D}(u)$ and pointer in $N_{\text{out}}(u, \mathcal{G})$ (resp. $N_{\text{in}}(u, \mathcal{G})$) for each vertex $u \in \mathcal{V}(\mathcal{G})$; during the traversal, in Line 10, at most one copy of each vertex is in Q , resulting in a space complexity of $\mathcal{O}(n)$.

Proof. (Proof of Theorem 4) Line 4 takes $\mathcal{O}(m)$ time to initialize $TCV_\tau(s, u)$ for each $\tau \in \mathcal{T}_{\text{in}}(u, \mathcal{G}_q)$ of vertex $u \in \mathcal{V}(\mathcal{G}_q) \setminus \{s, t\}$. Lines 5-6 take $\mathcal{O}(n)$ time to initialize the completed indicator and the pointer of each vertex $u \in \mathcal{V}(\mathcal{G}_q) \setminus \{s, t\}$. For each edge in $\mathcal{E}(\mathcal{G}_q)$, Lines 8-15 take $\mathcal{O}(1)$ time, Lines 17 and 19 take $\mathcal{O}(\theta)$ time as the number of vertices in each entry of $TCV(s, \cdot)$ is bounded by $\theta - 1$. Therefore, the total time complexity is $\mathcal{O}(n + \theta \cdot m)$, which is polynomial with respect to the input size since $\theta \leq |\mathcal{T}| \ll m$.

$TCV(s, \cdot)$ and $TCV(\cdot, s)$ have $\mathcal{O}(m)$ entries, and each entry has a length bounded by $\theta - 1$. Throughout the algorithm, we maintain the completed indicator and the pointer for each vertex $u \in \mathcal{V}(\mathcal{G}_q)$. Therefore, the total space complexity is $\mathcal{O}(n + \theta \cdot m)$.

Proof. (Proof of Theorem 5) The pointer initialization in Line 1 takes $\mathcal{O}(n)$ time and the pointer operations in Lines 8-10 take $\mathcal{O}(m)$ time in total. Line 3 iterates through each edge in $\mathcal{E}(\mathcal{G}_q)$ once and the intersection operation in Line 17 is performed at most m times. Each intersection operation takes $\mathcal{O}(\theta)$ time as the length of each entry in $TCV(s, \cdot)$ and $TCV(\cdot, s)$ is bounded by $\theta - 1$. Therefore, the total time complexity is $\mathcal{O}(n + \theta \cdot m)$, which is polynomial with respect to the input size, since $\theta \leq |\mathcal{T}| \ll m$. $\mathcal{G}_t$ takes $\mathcal{O}(n+m)$ space, therefore, the total space complexity is $\mathcal{O}(n+m)$.

Proof. (Proof of Theorem 6) Line 2 initializes a verified indicator for each edge, therefore takes $\mathcal{O}(m)$ time. Lines 3-5 can be implemented as a traversal in $\mathcal{G}_t$ from s (resp. t), which takes $\mathcal{O}(m)$ time. For each unverified edge $e(u, v, \tau)$, the Bidirectional DFS in Line 9 has a depth of $\theta - 1$ at most with a time complexity bounded by $\mathcal{O}(d_t^{\theta-1})$, Lines 11-18 takes $\mathcal{O}(d_t \cdot \theta)$ time to verify edges in the batch of paths. Therefore the total time complexity is $\mathcal{O}(m \cdot d_t^{\theta-1})$.

tspG takes $\mathcal{O}(n+m)$ space and the verified indicator for all edges takes $\mathcal{O}(m)$ space. The space for stacks $\mathcal{S}_v$ and $\mathcal{S}_e$ is in $\mathcal{O}(n)$. Thus, the total space complexity is $\mathcal{O}(n+m)$.

Proof. (Proof of Lemma 3) Sufficiency. If there exists a path $p_{[\tau_b, \tau_e]}^*(s, t)$ through $e(u, v, \tau)$, there exist two paths $p_{[\tau_b, \tau_i]}^*(s, u), p_{[\tau_j, \tau_e]}^*(v, t)$ s.t. $\tau_i < \tau < \tau_j$, and $\mathcal{V}(p_{[\tau_b, \tau_i]}^*(s, u)) \cap \mathcal{V}(p_{[\tau_j, \tau_e]}^*(v, t)) = \emptyset$. Based on the definition of time-stream common vertices, we have $TCV_{\tau_i}(s, u) \subseteq \mathcal{V}(p_{[\tau_b, \tau_i]}^*(s, u))$ and $TCV_{\tau_j}(v, t) \subseteq \mathcal{V}(p_{[\tau_j, \tau_e]}^*(v, t))$, thus, $TCV_{\tau_i}(s, u) \cap TCV_{\tau_j}(v, t) = \emptyset$.

Necessity. Consider $e(c, f, 4) \in \mathcal{E}(\mathcal{G}_q)$ in Fig. 3(c) as an counterexample. There only exist $\tau_i = 3$ and $\tau_j = 5$ satisfying the condition $\tau_i < 4 < \tau_j$, and we have $TCV_3(s, c) \cap TCV_5(f, t) = \emptyset$ as $TCV_3(s, c) = \{b, c\}$ and $TCV_5(f, t) = \{f\}$. However, there does not exist a temporal simple path $p_{[2, 7]}^*(s, t)$ through $e(c, f, 4)$. Therefore, the necessity is not established.

Proof. (Proof of Lemma 4) To prove Lemma 4, we first prove $\mathcal{P}_{[\tau_b, \tau_i]}^*(s, u) = \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$ by contradiction. Since $\tau_i \leq \tau$, $\mathcal{P}_{[\tau_b, \tau_i]}^*(s, u) \subseteq \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$. Suppose $\mathcal{P}_{[\tau_b, \tau_i]}^*(s, u) \neq \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$, this implies that there exists $p_{[\tau_b, \tau]}^*(s, u) \in \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$ such that $p_{[\tau_b, \tau]}^*(s, u) \notin \mathcal{P}_{[\tau_b, \tau_i]}^*(s, u)$. It is evident that such $p_{[\tau_b, \tau]}^*(s, u)$ contains an in-coming edge $e(v, u, \tau')$ of u where $\tau_i < \tau' \leq \tau$, which contradicts $\tau_i = \max\{\tau_i | \tau_i \in \mathcal{T}_{\text{in}}(u, \mathcal{G}_q), \tau_i \leq \tau\}$. In conclusion, $\mathcal{P}_{[\tau_b, \tau_i]}^*(s, u) = \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$, and we have $TCV_\tau(s, u) = TCV_{\tau_i}(s, u)$ following the definition of the time-stream common vertices. We omit the proof for $TCV_\tau(u, t) = TCV_{\tau_r}(u, t)$ as it follows a similar approach.

Proof. (Proof of Lemma 5) For each temporal path $p \in \mathcal{P}_{[\tau_b, \tau]}(s, u)$, there exists a temporal simple path $p^* \in \mathcal{P}_{[\tau_b, \tau]}^*(s, u)$ such that the path p^* contains all edges in the path p except those forming cycles. Then, we have $\mathcal{V}(p^*) \subseteq \mathcal{V}(p)$ to derive that $\mathcal{V}(p^*) \cap \mathcal{V}(p) = \mathcal{V}(p^*)$. Thus, $TCV_\tau(s, u) = \bigcap_{p^* \in \mathcal{P}_{[\tau_b, \tau]}^*(s, u) \text{ s.t. } t \notin \mathcal{V}(p^*)} \mathcal{V}(p^*) \setminus \{s\} = \bigcap_{p \in \mathcal{P}_{[\tau_b, \tau]}(s, u) \text{ s.t. } t \notin \mathcal{V}(p)} \mathcal{V}(p) \setminus \{s\}$. The proof for $TCV_\tau(u, t)$ follows the same approach.

Proof. (Proof of Theorem 7) Line 2 initializes the $\mathcal{D}(u)$ for each vertex $u \in \mathcal{V}(\mathcal{G}_q)$, which takes $\mathcal{O}(n)$ time. Lines 5-6 take $\mathcal{O}(n)$ time to initialize the completed indicator and the pointer of each vertex $u \in \mathcal{V}(\mathcal{G}_q) \setminus \{s, t\}$, and the pointer operations in Line 35 take $\mathcal{O}(m)$ time in total. For each edge in $\mathcal{E}(\mathcal{G}_q)$, Lines 23, 27 and 40 take $\mathcal{O}(\theta')$ time for union or intersection operation as the number of vertices in entry of $TCV(\cdot, t)$ is bounded by $\theta' - 1$. Therefore, the total time complexity is $\mathcal{O}(n + \theta' \cdot m)$. The algorithm maintains $\mathcal{D}$, TCV , completed indicator and the pointer for each vertex while

generating the new $\mathcal{G}'_q$ and $\mathcal{G}'_t$. Thus, the total space complexity remains the same as Algorithms 2-5.

Proof. (Proof of Lemma 8) If $TCV_{\tau_l}(s, u) \cap TCV_{\tau_r}(v, t) \neq \emptyset$, i.e., there exists a vertex w such that $w \in TCV_{\tau_l}(s, u)$ and $w \in TCV_{\tau_r}(v, t)$, then based on the definition of time-stream common vertices, $\forall p^* \in \mathcal{P}_{[\tau_b, \tau_l]}^*(s, u)$ s.t. $t \notin \mathcal{V}(p^*)$, $w \in p^*$. Since for all $\tau_b \leq \tau_i < \tau_l$, $\mathcal{P}_{[\tau_b, \tau_i]}^*(s, u) \subseteq \mathcal{P}_{[\tau_b, \tau_l]}^*(s, u)$, thus, $\forall p_i^* \in \mathcal{P}_{[\tau_b, \tau_i]}^*(s, u)$ s.t. $t \notin \mathcal{V}(p_i^*)$, $w \in p_i^*$, that is, we have $w \in TCV_{\tau_i}(s, u)$. Similarly, for all $\tau_r < \tau_j \leq \tau_e$, we have $w \in TCV_{\tau_j}(v, t)$. Therefore, $TCV_{\tau_i}(s, u) \cap TCV_{\tau_j}(v, t) \neq \emptyset$.

Proof. (Proof of Lemma 9) ($\Rightarrow$) If an edge $e(u, v, \tau) \in \mathcal{E}(\mathcal{G}_q)$, where $u \neq s$ and $v \neq t$, satisfies condition i), based on Lemma 3 and Lemma 8, such an edge will not be excluded as an unpromising edge. Therefore, it should be included in $\mathcal{G}_t$. If an edge $e(u, v, \tau) \in \mathcal{E}(\mathcal{G}_q)$, where $u = s$ or $v = t$, based on the condition ii) of Lemma 2, we can conclude that $e(u, v, \tau)$ belongs to tspG . Since $\mathcal{G}_t$ is the upper-bound graph of tspG , $e(u, v, \tau)$ must belong to $\mathcal{G}_t$.

($\Leftarrow$) If an edge $e(u, v, \tau)$ belongs to $\mathcal{G}_t$, there does not exist a vertex w appearing in all $p_{[\tau_b, \tau_i]}^*(s, u)$ and $p_{[\tau_j, \tau_e]}^*(v, t)$ where $\tau_i < \tau < \tau_j$. Based on the Definition 6, it is easy to derive that $e(u, v, \tau)$ satisfies condition i) s.t. $u \neq s$ and $v \neq t$. Since $e(u, v, \tau) \in \mathcal{G}_t$ must belong to $\mathcal{G}_q$, and we have discussed $w \neq s$ and $w \neq t$ for each edge in $\mathcal{G}_q$, therefore u can be the source vertex s or v can be the target vertex t .

Proof. (Proof of Lemma 10) Let $\tau_l = \max\{\tau'' \mid \tau'' \in \mathcal{T}_{in}(u, \mathcal{G}_q) \wedge \tau_b \leq \tau'' < \tau\}$, $\tau_r = \min\{\tau'' \mid \tau'' \in \mathcal{T}_{out}(v, \mathcal{G}_q) \wedge \tau < \tau'' \leq \tau_e\}$. For condition i), if there exists an edge $e(s, u, \tau') \in \mathcal{E}(\mathcal{G}_t)$ such that $\tau_b \leq \tau' \leq \tau_l < \tau$, then we have a temporal simple path $p_{[\tau_b, \tau_l]}^*(s, u) = \langle e(s, u, \tau') \rangle$, therefore $TCV_{\tau_l}(s, u) = \{u\}$. Since $e(u, v, \tau) \in \mathcal{G}_t$, according to Lemma 9, $TCV_{\tau_l}(s, u) \cap TCV_{\tau_r}(v, t) = \emptyset$, that is, $u \notin TCV_{\tau_r}(v, t)$. Then, based on the definition of $TCV_{\tau_r}(v, t)$, there exists a temporal simple path $p_{[\tau_r, \tau_e]}^*(v, t)$ that does not pass through s and u . Therefore, a temporal simple path $p_{[\tau_b, \tau_e]}^*(s, t)$ that includes $e(u, v, \tau)$ can be formed. We omit the proof for condition ii) as it is similar to that of condition i).

Proof. (Proof of Lemma 16) From Line 9 of Algorithm 7, the condition $\mathcal{D}^{[\tau_b, \tau_j]}(t) - 1 \leq \tau_i$ guarantees that each edge $e(u, v, \tau) \in \mathcal{E}(\mathcal{G}_q^{[\tau_b, \tau_j]})$ satisfies $\tau \leq \tau_i$. Consequently, for any $\tau \in [\tau_i, \tau_j]$, shrinking the time interval from $[\tau_b, \tau_j]$ to $[\tau_b, \tau]$ does not remove any edge from $\mathcal{G}_q^{[\tau_b, \tau_j]}$. Therefore, the latest departure times $\mathcal{D}$ remain unchanged, yielding $\mathcal{G}_q^{[\tau_b, \tau]} =$

Algorithm 9: BiDirSearch

Input : an edge $e(u, v, \tau) \in \mathcal{E}(\mathcal{G}_t)$
Output : the existence of $p_{[\tau_b, \tau_e]}^*(s, t)$ through $e(u, v, \tau)$

```

1  $S_v.push(u)$ ,  $S_v.push(v)$ ;
2 if  $\tau - \tau_b > \tau_e - \tau$  then  $dir = f$  else  $dir = b$ ;
3  $F \leftarrow false$ ,  $B \leftarrow false$ ;
4 if  $Search(u, v, \tau, dir, F, B)$  then return true;
5 else return false;
6 Procedure  $Search(u, v, \tau, dir, F, B)$ :
7   if  $dir = f$  then  $S_v.push(v)$  else  $S_v.push(u)$ ;
8    $S_e.push(e(u, v, \tau))$ ;
9   if  $v = t \vee u = s$  then
10     if  $v = t$  then  $dir = b$ ,  $F \leftarrow true$ ;
11     if  $u = s$  then  $dir = f$ ,  $B \leftarrow true$ ;
12     if  $F \wedge B$  then return true;
13      $e(u_0, v_0, \tau_0) \leftarrow$  the first edge in  $S_e$ ;
14     if  $Search(u_0, v_0, \tau_0, dir, F, B)$  then return true;
15      $S_v.pop()$ ,  $S_e.pop()$ ;
16   return false;
17 if  $dir = f$  then
18   for each  $(w_i, \tau_i) \in N_{out}(v, \mathcal{G}_t)$  do
19     if  $\tau_i \leq \tau$  then break;
20     if  $w_i \in S_v$  then continue;
21     if  $Search(v, w_i, \tau_i, dir, F, B)$  then return true;
22 else
23   for each  $(w_i, \tau_i) \in N_{in}(u, \mathcal{G}_t)$  do
24     if  $\tau_i \geq \tau$  then break;
25     if  $w_i \in S_v$  then continue;
26     if  $Search(w_i, u, \tau_i, dir, F, B)$  then return true;
27  $S_v.pop()$ ,  $S_e.pop()$ ;
28 return false;
29 end

```

$\mathcal{G}_q^{[\tau_b, \tau_j]}$. Because the quick upper-bound graphs are identical, the time-stream common vertices TCV are also unchanged, which leads to $\mathcal{G}_t^{[\tau_b, \tau]} = \mathcal{G}_t^{[\tau_b, \tau_j]}$. Finally, the EEV phase, executed on the same tight upper-bound graph, produces the same temporal simple path graph, i.e., $\text{tspG}_{[\tau_b, \tau]}(s, t) = \text{tspG}_{[\tau_b, \tau_j]}(s, t)$.

Appendix B

BiDirSearch. With the proposed optimizations, our implementation of the bidirectional DFS is detailed in Algorithm 9. In Line 2, we determine whether to perform a forward or a backward search first. Then, Line 4 starts the *search* procedure with two flags F (resp. B), indicating whether a forward path (resp. backward path) has been found in the current search, initially set to *false*. The *search* procedure recursively searches for a temporal simple path in its given direction. Once a forward (resp. backward) path is completed, it sets the corresponding flag F (resp. B) and toggles the search

Table 3: Summary of Notations

Notation	Description
$\mathcal{G}/\mathcal{S}$	a directed temporal graph/an induced subgraph of $\mathcal{G}$
$N_{out/in}(u, \mathcal{S})$	the out/in-neighbors of u in $\mathcal{S}$
$\mathcal{T}_{out/in}(u, \mathcal{S})$	the set of all distinct timestamps in the out/in-neighbors of u in $\mathcal{S}$
$\mathcal{P}, \mathcal{P}_{[\tau_b, \tau_e]}(s, t)$	the set of all temporal paths $p_{[\tau_b, \tau_e]}(s, t)$
$\mathcal{P}^*, \mathcal{P}_{[\tau_b, \tau_e]}^*(s, t)$	the set of all temporal simple paths $p_{[\tau_b, \tau_e]}^*(s, t)$
$tspG, tspG_{[\tau_b, \tau_e]}(s, t)$	the temporal simple path graph from s to t within $[\tau_b, \tau_e]$
$\mathcal{G}_q/\mathcal{G}_q^{[\tau_i, \tau_j]}$	the quick upper-bound graph of $tspG/tspG_{[\tau_i, \tau_j]}(s, t)$
$\mathcal{G}_t/\mathcal{G}_t^{[\tau_i, \tau_j]}$	the tight upper-bound graph of $tspG/tspG_{[\tau_i, \tau_j]}(s, t)$
$\mathcal{A}(u)/\mathcal{A}^{[\tau_i, \tau_j]}(u)$	the earliest arrival time from s to u in $tspG/tspG_{[\tau_i, \tau_j]}(s, t)$
$\mathcal{D}(u)/\mathcal{D}^{[\tau_i, \tau_j]}(u)$	the latest departure time from u to t in $tspG/tspG_{[\tau_i, \tau_j]}(s, t)$
$TCV_\tau(s, u)/TCV_\tau(u, t)$	the time-stream common vertices of $\mathcal{P}_{[\tau_b, \tau]}^*(s, u)/\mathcal{P}_{[\tau, \tau_e]}^*(u, t)$ where $\tau_b < \tau < \tau_e$
$TCV_\tau^{[\tau_i, \tau_j]}(s, u)/TCV_\tau^{[\tau_i, \tau_j]}(u, t)$	the time-stream common vertices of $\mathcal{P}_{[\tau_i, \tau]}^*(s, u)/\mathcal{P}_{[\tau, \tau_j]}^*(u, t)$ where $\tau_i < \tau < \tau_j$

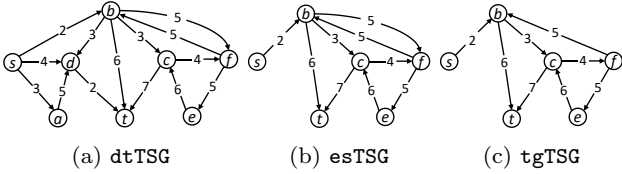


Fig. 16: Upper-bound graphs of baselines within $[2, 7]$ direction in Lines 10-11. If the path in only one direction is completed, the *search* procedure will continue in the other direction and will terminate upon finding paths in both directions in Line 12. In Lines 18 and 23, the neighbor exploration order is determined based on the current search direction.

Appendix C

Example. (Example of Baseline methods)

Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a). Suppose the source vertex is s , the target vertex is t , and the time interval is $[2, 7]$. In Figs. 16(a)-16(c), we illustrate three upper-bound graphs obtained by three baseline algorithms.

Example. (Example of Definition 5) Given the temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ in Fig. 1(a), a source vertex s , a target vertex t and a time interval $[2, 7]$. For the vertex $f \in \mathcal{V} \setminus \{s, t\}$, $\mathcal{P}_{[2, 7]}(s, f) =$

$\{\langle e(s, b, 2), e(b, f, 5) \rangle, \langle e(s, b, 2), e(b, c, 3), e(c, f, 4) \rangle\}$, we have $\mathcal{A}(f) = \min\{4, 5\} = 4$. Similarly, $\mathcal{P}_{[2, 7]}(f, t) = \{\langle e(f, e, 5), e(e, c, 6), e(c, t, 7) \rangle, \langle e(f, b, 5), e(b, t, 6) \rangle\}$, we have $\mathcal{D}(f) = 5$.

Example. (Example of Algorithm 2) Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a), the precomputed polarity time in Figs. 3(a)-3(b), the resulting quick upper-bound graph $\mathcal{G}_q$ is shown in Fig. 3(c). We exclude $e(s, a, 3)$ since $\mathcal{D}(a) = -\infty < 3$, $e(d, t, 2)$ as $\mathcal{A}(d) = 3 > 2$.

Example. (Example of Algorithm 3) Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a). For the query of s to t within $[2, 7]$, the earliest arrival time of each vertex is shown in Fig. 3(a). As we process s , its out-neighbors $\{(b, 2), (a, 3), (d, 4)\}$ are checked. We add b, a, d to Q and update their earliest arrival time to 2, 3, 4, respectively. Next, $\mathcal{A}(d)$ is updated to 3 when b is being processed. Note that, we do not add d to Q since it is currently in Q . Then, a is processed without changes to $\mathcal{A}(d)$, since $5 > 3 = \mathcal{A}(d)$.

Example. (Example of Algorithm 4) Consider $\mathcal{G}_q$ in Fig. 3(c), the time-stream common vertices $TCV(s, \cdot)$ and $TCV(\cdot, t)$ are shown in Figs. 4(a)-4(b). Take the computation of $TCV_5(f, t)$ as an example. When $e(f, e, 5)$ is scanned, we have $i = 1$, $\tau_i = 5$, $\tau_j = 6$. Since $TCV_5(f, t) = \text{null}$, we are starting to process the first entry in $TCV(f, t)$ and we obtain $TCV_5(f, t) = TCV_6(e, t) \cup \{f\} = \{c, e\} \cup \{f\} = \{c, e, f\}$ in Line 17. After that, we process $e(f, b, 5)$ with $i = 1$, $\tau_i = 5$, $\tau_j = 6$. Since $TCV_5(f, t) \neq \text{null}$ and $\tau = 5 = \tau_i$, we continue the iterative intersection process for the first entry in $TCV_5(\cdot, t)$ and update $TCV_5(f, t)$ to $\{c, e, f\} \cap (TCV_6(b, t) \cup \{f\}) = \{f\}$. Then, the pruning condition in Line 22 is satisfied, and we mark f as completed.

Example. (Example of Algorithm 5) Given $\mathcal{G}_q$ in Fig. 3(c), the precomputed $TCV(s, \cdot)$ and $TCV(\cdot, t)$ in Figs. 4(a)-4(b). The resulting tight upper-bound graph $\mathcal{G}_t$ is shown in Fig. 4(c). Take $e(c, f, 4)$ as an example, it is contained in $\mathcal{G}_t$ as $TCV_3(s, c) \cap TCV_5(f, t) = \emptyset$.

Example. (Example of Algorithm 7) Consider the quick upper-bound graph $\mathcal{G}_q$ within time interval $[1, 10]$, as shown in Fig. 17(a). When the query end time is shifted earlier to 9, the incrementally generated upper-bound graphs $\mathcal{G}_q'$ and $\mathcal{G}_t'$, together with the updated intermediate information (i.e., polarity times $\mathcal{A}(\cdot)$ and $\mathcal{D}(\cdot)$, time-stream common vertices $TCV(s, \cdot)$ and $TCV(\cdot, t)$), are illustrated in Figs. 17(b)-17(f). As an example, consider the scan of the edge $e(e, c, 8)$. Since the condition $\mathcal{D}(c) = 9 > 8$

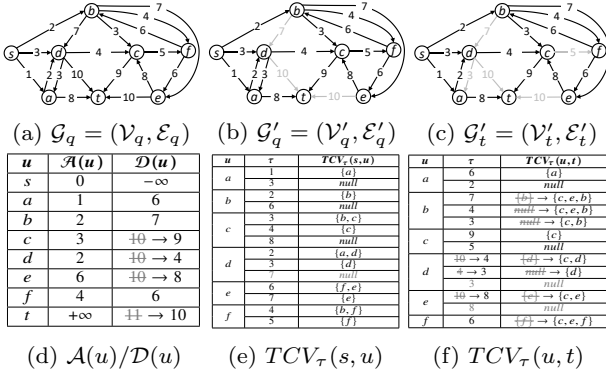


Fig. 17: Incremental Upper-bound Graph Generation

holds, the edge can be added to $\mathcal{G}'_q$ in Line 11. Moreover, because $\mathcal{D}(e) = -\infty < 8$, this edge is identified as the first outgoing edge of vertex e processed in $\mathcal{G}'_q$. Accordingly, we update $\mathcal{D}(e)$ to 8 and compute $TCV_8(e, t) = TCV_9(c, t) \cup \{e\} = \{c\} \cup \{e\} = \{c, e\}$ as shown in Lines 22–23. Finally, with $k = 2$ and $\tau_k = 7$, the TCV intersection check yields $TCV_7(s, e) \cap TCV_9(c, t) = \emptyset$, thereby confirming that $e(e, c, 8)$ should also be included in $\mathcal{G}'_t$.

Appendix D

Experiment Setup

Algorithms. We implement and evaluate the following four algorithms for the temporal simple path graph construction problem (Problem 1). Three baselines i.e., EPdtTSG, EPesTSG, EPTgTSG, whose details can be found in Section 3.1. VUG: Algorithm 1 proposed in Section 3.2 with all the optimized strategies.

In addition, we implement and evaluate the following three algorithms for the batch temporal simple path graph construction problem by introducing the end-time tolerance window (Problem 2). Two baselines i.e., BVUGe-Base and BVUGe-tdBase, whose details can be found in Section 6. BVUGe-tdOPT: Algorithm 8 proposed in Section 6.3 with all the optimized strategies.

We also implement and evaluate the following three algorithms for the batch temporal simple path graph construction problem by introducing the begin-time tolerance window (Problem 3). Two baseline methods: BVUGb-Base and BVUGb-tdBase, as discussed in Section 6. BVUGb-tdOPT: proposed in Section 6.3, which adapts all optimized strategies from BVUGe-tdOPT to the begin-time tolerance window setting.

Datasets. We employ 10 real-world temporal graphs in our experiments. Details of these datasets are summarized in Table 4. $|\mathcal{T}|$ is the number of snapshots, d_{max} is the maximum degree in a dataset. Among these

datasets, D1-D4, D7, D9 are downloaded from SNAP², and D5, D6, D8, D10 are downloaded from KONECT³. All these datasets are publicly available.

Parameters and workloads. For each dataset, we generate 1000 random queries with different source vertices s , target vertices t , time intervals $[\tau_b, \tau_e]$ where s can temporally reach t within $[\tau_b, \tau_e]$ and report the total query time on each dataset. We conduct the experiment by varying $\tau_e - \tau_b + 1$ (i.e., θ), whose default values are set following existing works about path problems in temporal graphs (e.g., [20, 26, 29]). Details are shown in the last column of Table 4. For those algorithms that cannot finish within 12 hours, we set them as **INF**. All the programs are implemented in standard C++. All the experiments are performed on a server with Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz and 256G RAM.

Supplementary experiments for temporal simple path graph construction

Supplement to Exp-2: Response time by varying θ on the other datasets. In Fig. 18, we report the total response time of VUG for 1000 queries by varying θ , along with the baseline algorithms as comparison. On each dataset, the response time of VUG grows modestly, which confirms the scalability of our proposed methods. For example, on the largest dataset D10, three baseline methods cannot finish queries within 12 hours when $\theta = 26$, while the total response time of VUG only increases by a factor of 1.2 when θ increases from 23 to 27.

Supplement to Exp-3: Response time by varying vertex pairs w.r.t. degree on the other datasets. In Fig. 19, we report the total response time of VUG for 1000 queries by varying vertex pairs across different out-degree and in-degree groups, along with the baselines as comparison. On each dataset, the response time of VUG remains relatively stable from high-degree to low-degree vertex pairs. For example, on the dataset D10, the three baselines cannot finish the queries within 12 hours for the top 40% degree groups, while the response time of VUG increases only from 122 seconds to 148 seconds as the vertex pairs vary from the lowest-degree group to the highest-degree group.

Supplement to Exp-4: Response time by varying vertex pairs w.r.t. distance on the other datasets. In Fig. 20, we report the total response time of VUG for 1000 queries by varying the distance between the source and target vertices, along with the baselines

² <http://snap.stanford.edu>

³ <http://konect.cc/networks/>

Table 4: Statistics of datasets

Dataset	$ \mathcal{V} $	$ \mathcal{E} $	$ \mathcal{T} $	d_{max}	θ
D1 (email-Eu-core)	1,005	332,334	803	9,782	10
D2 (sx-mathoverflow)	88,581	506,550	2,350	5,931	20
D3 (sx-askubuntu)	159,316	964,437	2,613	8,729	20
D4 (sx-superuser)	194,085	1,443,339	2,773	26,996	20
D5 (wiki-ru)	457,018	2,282,055	4,715	188,103	25
D6 (wiki-de)	519,404	6,729,794	5,599	395,780	25
D7 (wiki-talk)	1,140,149	7,833,140	2,320	264,905	20
D8 (flickr)	2,302,926	33,140,017	196	34,174	10
D9 (sx-stackoverflow)	6,024,271	63,497,050	2,776	101,663	20
D10 (wikipedia)	2,166,670	86,337,879	3,787	218,465	25

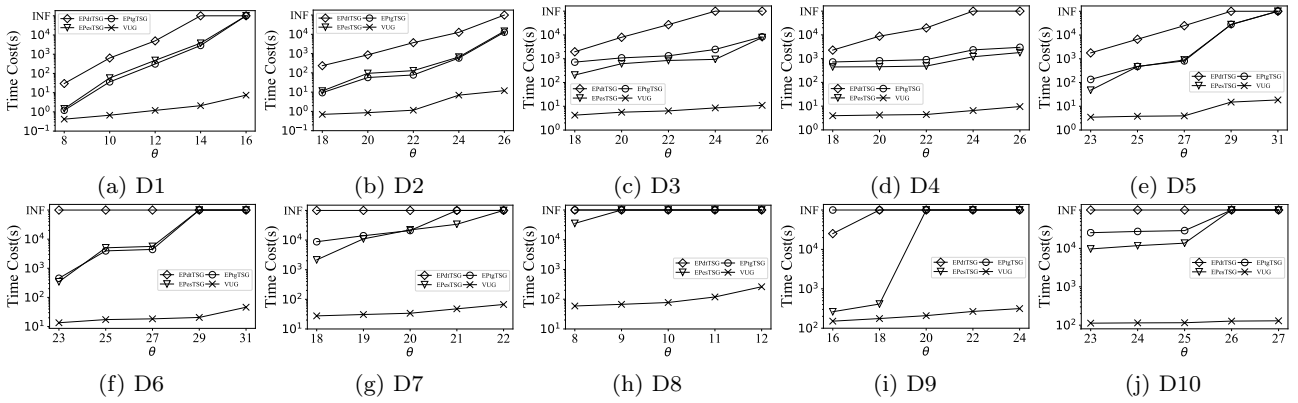
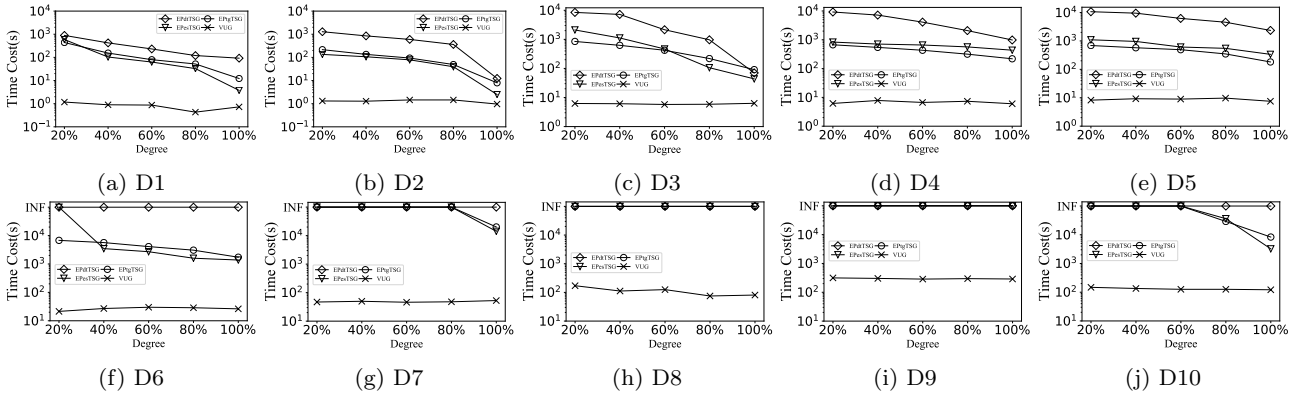
Fig. 18: Response time by varying θ on all other datasets

Fig. 19: Response time by varying vertex pairs w.r.t. degree on all other datasets

as comparison. On each dataset, the response times of the three baselines vary substantially with the distance between vertex pairs, whereas the response time of VUG remains relatively stable. For example, on the dataset D7, EPTgTSG cannot finish the queries within 12 hours when $d = 6$ but completes them in 3.8 hours when $d = 10$, while the response time of VUG varies by no more than 35 seconds across the five distance settings.

Supplement to Exp-8: Evaluation of output in $tspG$ by varying θ on the other datasets. In Fig. 21, we report the average numbers of vertices and edges in $tspG$, as well as the average number of temporal simple paths and the average total number of edges across these paths, for 1000 queries by varying θ . For every dataset and θ setting, the average number of temporal simple paths in $tspG$ and the average total number of edges across these paths far exceed the average num-

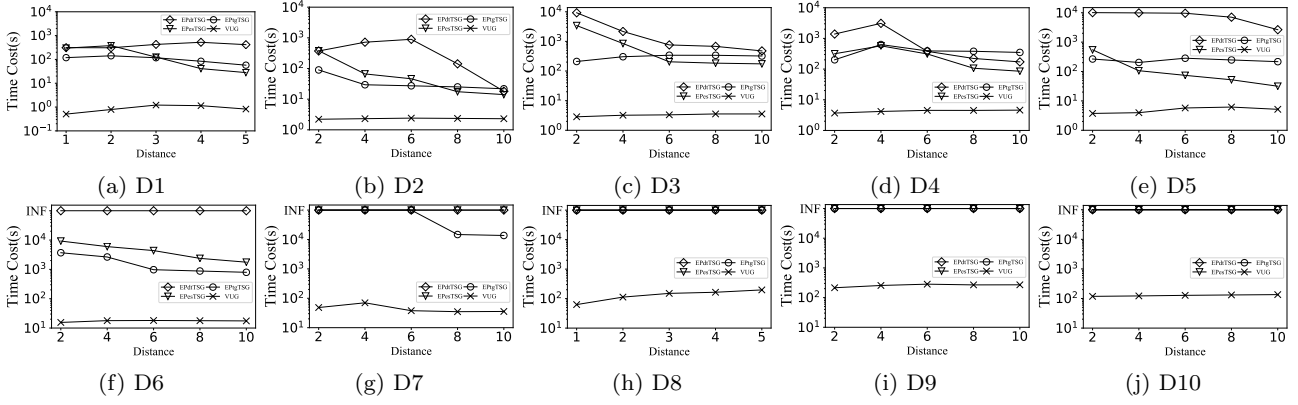


Fig. 20: Response time by varying vertex pairs w.r.t. distance on all other datasets

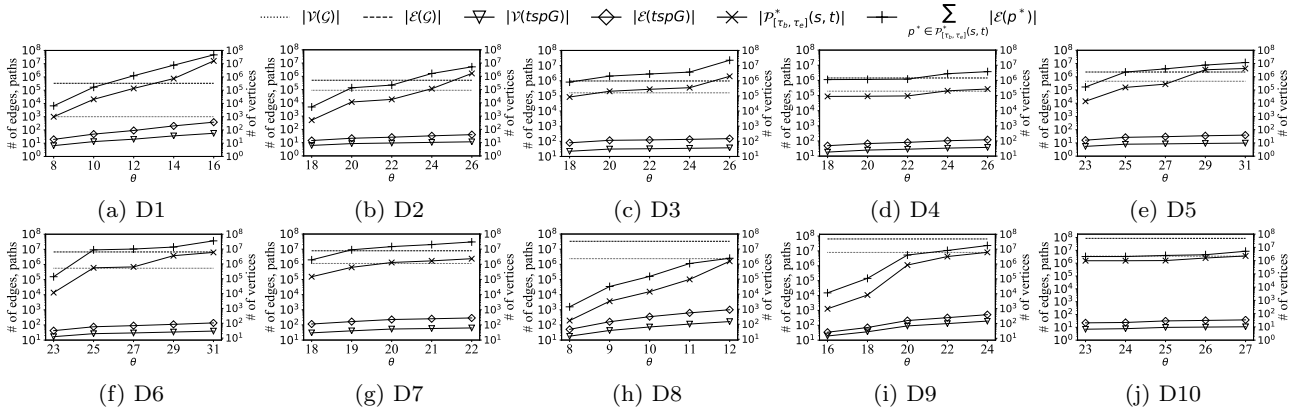
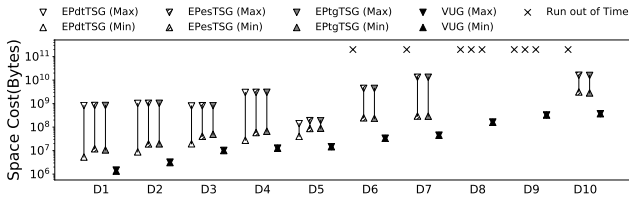
Fig. 21: Evaluation of output in *tspG* by varying θ on the other datasets

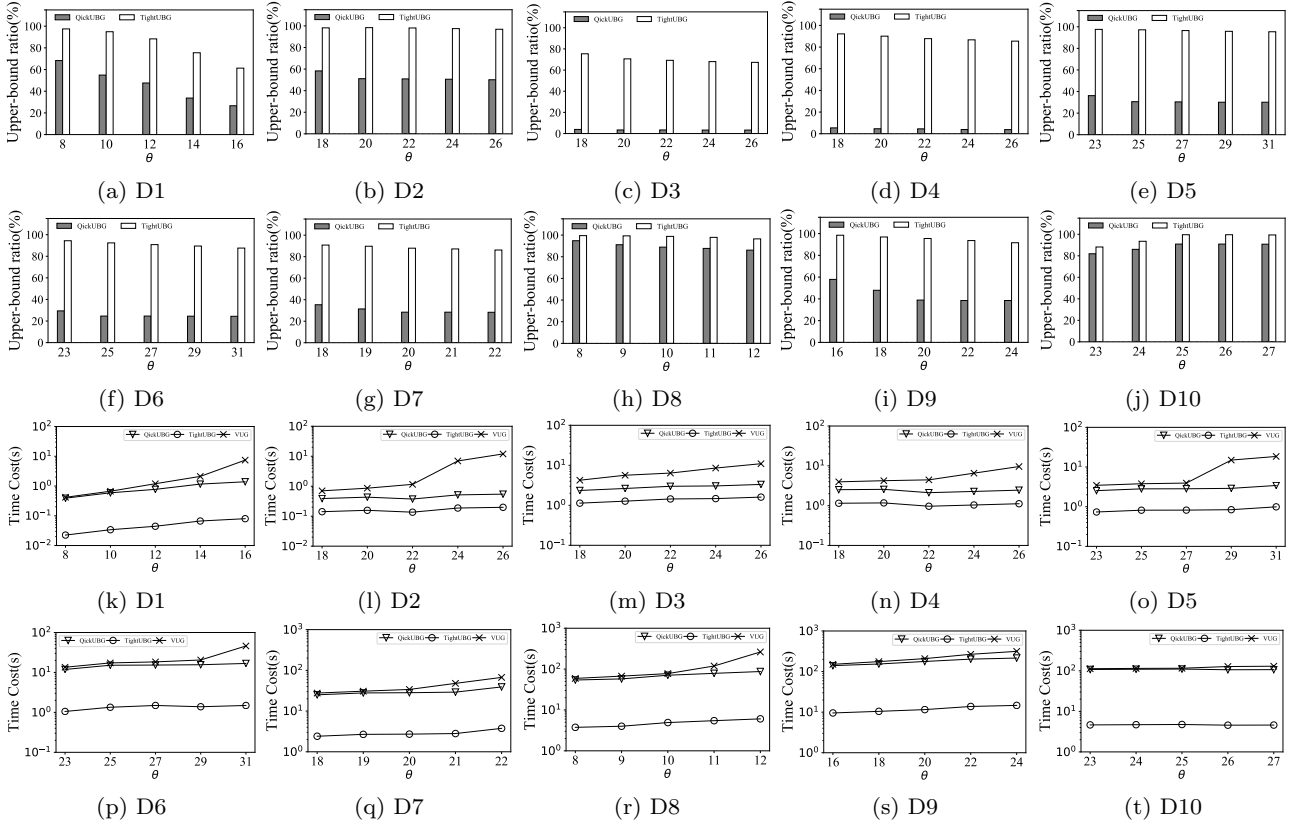
Fig. 22: Space consumption comparison

bers of vertices and edges. For example, when $\theta = 27$ on the dataset D10, the generated *tspG* contains only 10.18 vertices and 37.95 edges on average, while the average number of temporal simple paths exceeds 3.65 million and the average total number of edges across these paths reaches 8.51 million.

Supplementary Exp-1: Space consumption comparison. Fig. 22 reports the space consumption of VUG and the three baseline algorithms. For each algorithm, we report its maximum and minimum space consumption among 1000 queries on each dataset under the default settings. As observed, VUG consistently consumes less space compared to baseline algorithms, attributed to the fact that it avoids enumerating and explicitly

storing all temporal simple paths. In addition, VUG maintains a stable space consumption across different queries, regardless of the changing number of temporal simple paths, whereas the baselines exhibit exponential differences between their maximum and minimum space consumption. This supports our theoretical analysis in Theorems 2-6, that is, the space complexity of VUG is linear in the number of vertices and edges in the datasets.

Supplementary Exp-2: Evaluation of VUG-UGG by varying θ . Fig. 23 reports the response time and the upper-bound ratio for QuickUBG and TightUBG on each dataset. As shown, the upper-bound ratio slightly decreases with increasing θ in most cases, i.e., D1 (97.5%-61.3%), D3 (75.4%-67.3%), D4 (92.2%-85.5%), D5 (97.6%-95.4%), D6 (94.4%-87.6%), D7 (90.8%-86.2%), D8 (99.5%-96.5%), D9 (98.5%-91.6%) within presented ranges of θ , due to the introduction of more redundant edges that can not be pruned. However, in some cases, the ratio slightly increases, i.e., D2 (98.2%-98.4%) as θ increases from 18 to 20 and D10 (88.2%-99.6%) as θ increases from 23 to 25, because certain previously

Fig. 23: Evaluation of VUG-UGG by varying θ

redundant edges can be included in newly established temporal simple paths from s to t within a larger time interval and thus belong to $tspG$. Meanwhile, the overhead of QuickUBG and TightUBG remains stable across various settings of θ on all the datasets, which confirms the scalability. For example, on the largest dataset D10, when θ increases from 23 to 27, the time overhead of VUG increases from 113 seconds to 130 seconds. Within this, the overhead of QuickUBG ranges from 105 seconds to 107 seconds, and that of TightUBG ranges from 4.3 seconds to 4.6 seconds.

Supplementary experiments for batch temporal simple path graph construction

Supplement to Exp-11: Batch response time comparison by varying vertex pairs w.r.t. degree on the other datasets. In Fig. 24, we report the total response times of BVUGe-Base, BVUGe-tdBase, and BVUGe-tdOPT, as well as those of BVUGb-Base, BVUGb-tdBase, and BVUGb-tdOPT, for the five query sets in different degree groups on the other datasets. On each dataset, the response times of the three algorithms in each setting remain relatively stable across the degree

groups, and BVUGe-tdOPT and BVUGb-tdOPT have lower response times than their corresponding baselines. For example, in the 20% degree group on D9, the response times of BVUGe-Base, BVUGe-tdBase, and BVUGe-tdOPT are 2481, 2700, and 370 seconds, respectively.

Supplement to Exp-12: Batch response time comparison by varying vertex pairs w.r.t. distance on the other datasets. In Fig. 25, we report the total response times of BVUGe-Base, BVUGe-tdBase, and BVUGe-tdOPT, as well as those of BVUGb-Base, BVUGb-tdBase, and BVUGb-tdOPT, for the five query sets at different distances on the other datasets. On each dataset, all the three algorithms maintain a relatively stable response time across the five distances. BVUGe-tdOPT and BVUGb-tdOPT achieve the lowest response times in the end-time and begin-time tolerance window settings, respectively. For example, when the distance is 10 on D10, the response times of BVUGb-Base, BVUGb-tdBase, and BVUGb-tdOPT are 680, 708, and 140 seconds, respectively.

Supplementary Exp-1: Evaluation the performance of IncrementalUGG. In this experiment, we evaluate the performance of upper-bound graph generation (UGG) component across the three batch construc-

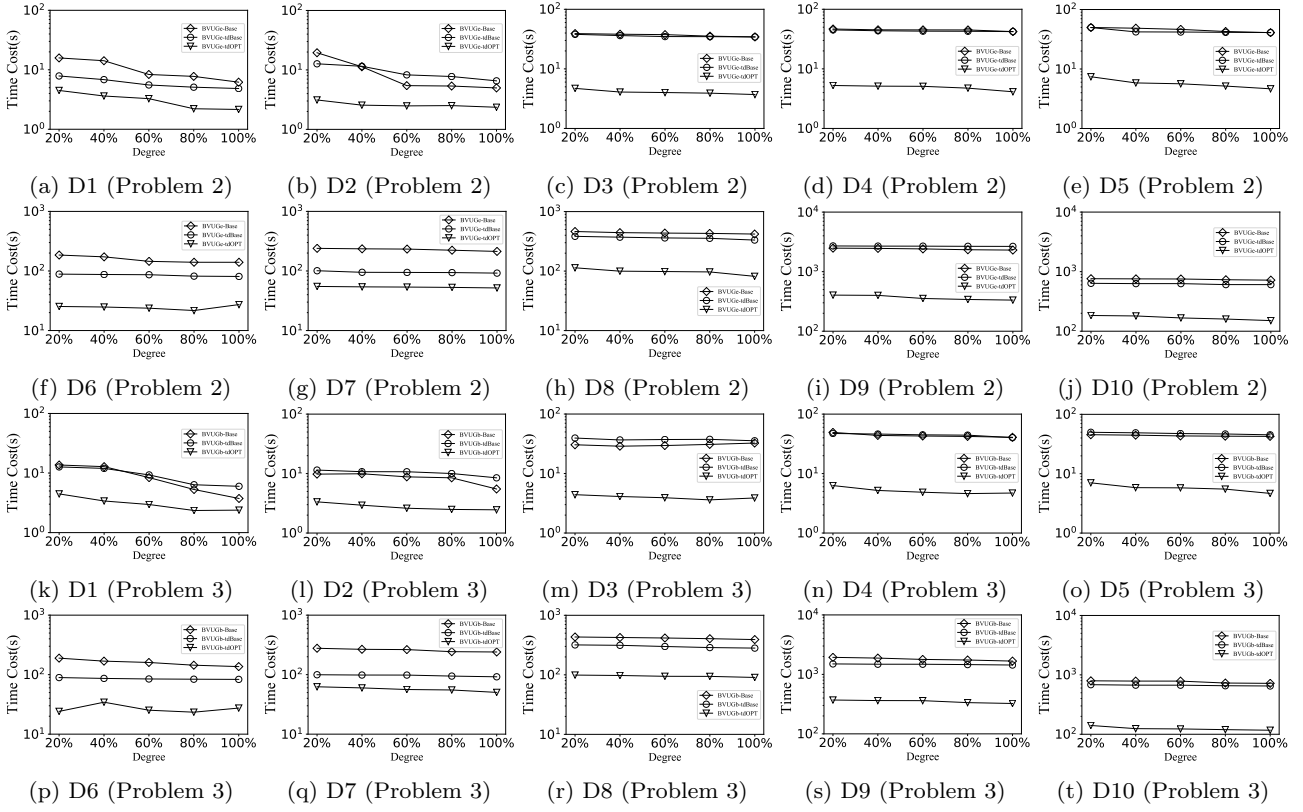


Fig. 24: Batch response time comparison by varying vertex pairs w.r.t. degree

tion algorithms. To isolate and quantify the advantages of our proposed incremental upper-bound graph update technique, we report the total time for generating the upper-bound graphs across the remaining $W - 1$ intervals for 1000 queries on each dataset. Note that, the cost for the initial (i.e., largest) interval is omitted here as it is identical for all the methods. Fig. 26(a) reports this execution time for BVUGe-Base, BVUGe-tdBase, and BVUGe-tdOPT. As observed, BVUGe-tdBase consistently outperforms BVUGe-Base in the UGG component, as the progressive shrinking strategy in BVUGe-tdBase effectively prunes the search space, thereby reducing the computational effort required for upper-bound graph construction. More importantly, BVUGe-tdOPT achieves a significant speedup over baselines in the UGG component by two orders of magnitude on most datasets, which demonstrates the efficiency of our incremental upper-bound graph update technique. Similarly, Fig. 26(b) illustrates the performance of UGG component for BVUGb-Base, BVUGb-tdBase and BVUGb-tdOPT. The experimental trends are consistent with those observed in end-time tolerance window setting, with BVUGb-tdOPT achieving significant speedups in the UGG component across all datasets. For example, on the largest dataset D10, the overhead for generating the re-

maining $W - 1$ upper-bound graphs across 1000 queries under BVUGb-Base and BVUGb-tdBase is 428 seconds and 400 seconds, respectively, while BVUGb-tdOPT reduces this overhead to merely 1.3 seconds.

Supplementary Exp-2: Evaluation of the performance of EEV. In this experiment, we evaluate the performance of Escaped Edges Verification (EEV) component across the three batch construction algorithms. Fig. 27(a) reports the total response time of the EEV component for BVUGe-Base, BVUGe-tdBase, and BVUGe-tdOPT for 1000 queries across all datasets. As we can see, BVUGe-tdBase outperforms BVUGe-Base in the EEV component since it operates on the $\mathcal{G}_t$ derived from the preceding obtained $tspG$, which reduces the search space for bidirectional DFS. BVUGe-tdOPT also outperforms BVUGe-Base by reusing results across intervals and skipping EEV when the updated graph structures remain unchanged. Notably, although the search space of BVUGe-tdOPT is slightly larger than that of BVUGe-tdBase, the result-reuse strategy leads to greater efficiency. Similarly, Fig. 27(b) reports the total response time of EEV component for BVUGb-Base, BVUGb-tdBase, and BVUGb-tdOPT for 1000 queries across all datasets. The experimental trends are consistent with those observed in end-time tolerance win-

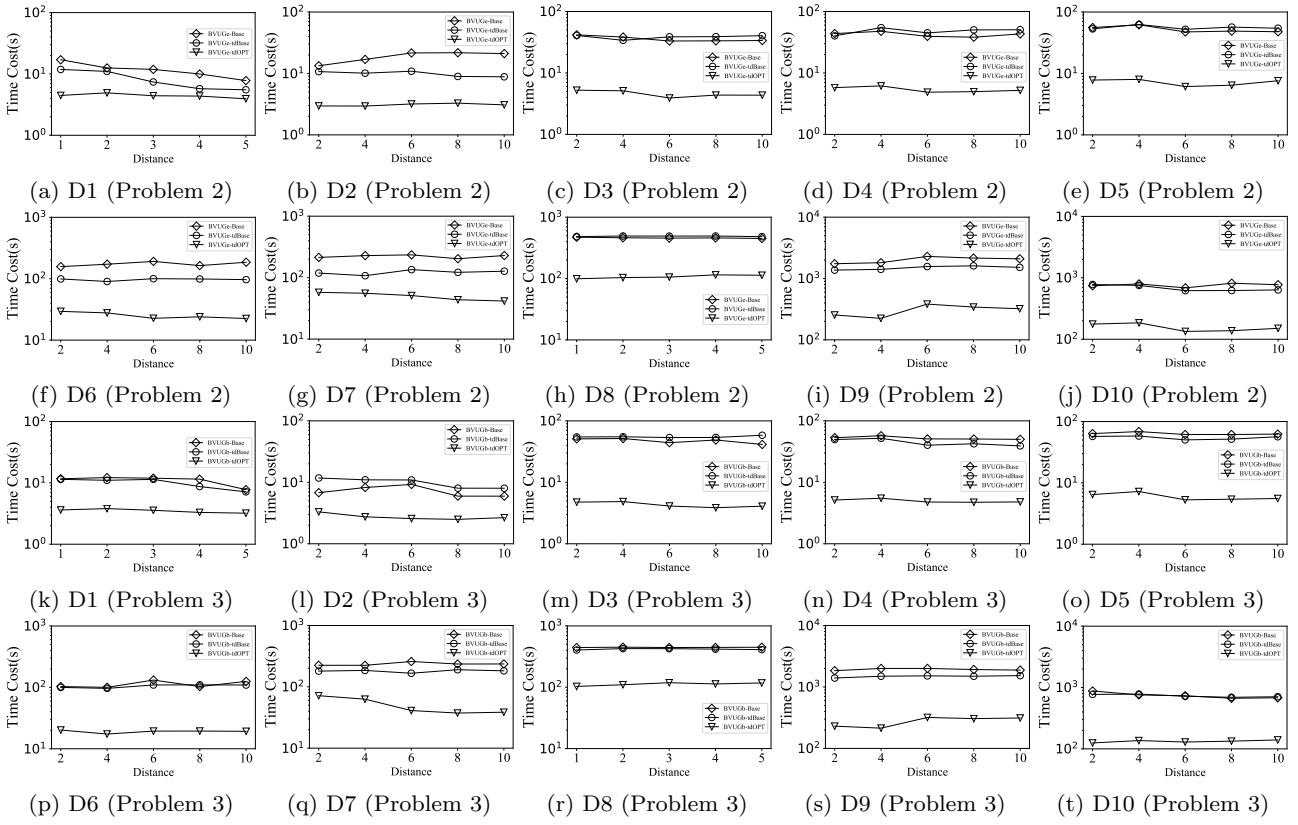


Fig. 25: Batch response time comparison by varying vertex pairs w.r.t. distance

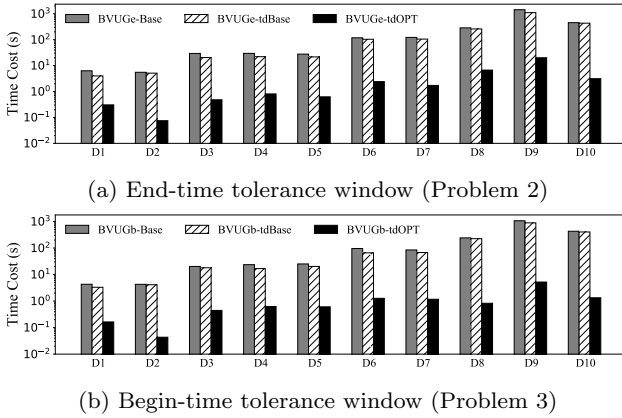


Fig. 26: Evaluation of IncrementalUGG

dow setting, with BVUGb-tdOPT achieving the best performance in the EEV component across all datasets. For example, on the dataset D10, the total response time of EEV component across 1000 queries under BVUGb-Base and BVUGb-tdBase is 12.5 seconds and 3.7 seconds, respectively, while BVUGb-tdOPT takes 2.1 seconds.

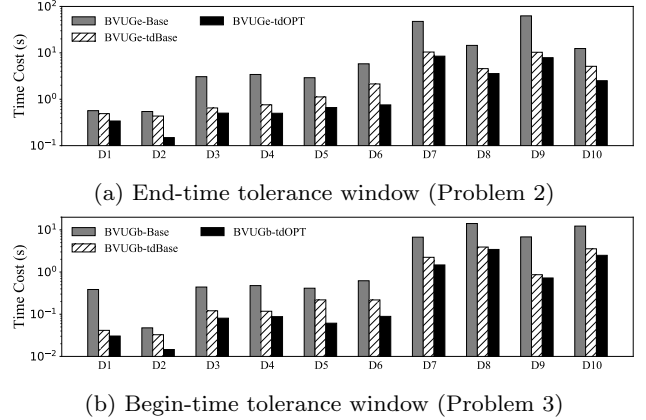


Fig. 27: Evaluation of EEV